ELEMENTI RAZVOJA SOFTVERA

Način polaganja i kriterijumi bodovanja

Ovaj dokument definiše strukturu polaganja predmeta, bodovanje projektnog rada, kontrolne tačke, organizaciju odbrane, teorijska pitanja, popravni termin i pravila formiranja konačne ocene.

71	41	51
maksimum na projektnom delu	minimum projektnog dela	upis ocene bez usmenog

1. Struktura polaganja i formiranje ocene

Provera znanja na predmetu zasniva se na kontinuiranom radu na računarskim vežbama, izradi i odbrani projektnog zadatka i usmenom delu ispita, u skladu sa uslovima navedenim u nastavku.

- Računarske vežbe i projektni zadatak: **najviše 71 poen**.
- Minimum za uspešno položen projektni deo: **41 poen**.
- Sa **51 ili više poena** na projektnom delu student može da prihvati ocenu formiranu na osnovu ostvarenih poena, **bez polaganja usmenog dela ispita**.
- Ukoliko student koji ima **najmanje 51 poen** sa projektnog dela prijavi ispit, a ne pristupi usmenom delu ispita, **upisuje se ocena koja odgovara broju poena ostvarenih na projektu**.
- Sa **41 do 50 poena** na projektnom delu student **pristupa usmenom delu ispita**. Za pozitivnu konačnu ocenu potrebno je dostići **najmanje 51 poen**.
- Rezultat usmenog dela može uvećati ukupan broj poena za **najviše 30**, odnosno umanjiti ga za **najviše 20 poena**.
- Konačan rezultat iskazuje se na skali od **0 do 100 poena**. Rezultat veći od 100 evidentira se kao **100 poena**.
- Poeni ostvareni na projektnom delu važe **do kraja tekuće akademske godine** i **ne prenose se u narednu akademsku godinu**!

Projektni deo	Minimum projekta	Upis bez usmenog	Usmeni deo
do 71 poen	41 poen	od 51 poena	-20 do +30

2. Bodovanje projektnog zadatka

Projektni zadatak se vrednuje po tematskim celinama i kontrolnim tačkama. Kontrolne tačke su sastavni deo ukupnog projektnog rezultata i **ne dodaju se preko maksimuma od 71 poena**.

Oblast	Poeni
Tapiz Boards, Agile i Scrum - organizacija i praćenje razvojnog procesa	10
SOLID principi i Clean Architecture	16
Poslovna logika i automatizovano testiranje	5
Kontrolna tačka 1 - osnovna implementacija (termin Vežbe 4)	10
Interoperabilnost i usklađenost ugovornih strana	10
Pravilna i kontrolisana upotreba AI alata u izradi projekta	10

Oblast	Poeni
Kontrolna tačka 2 - završna demonstracija (termin Vežbe 8)	10
UKUPNO	71

2.1. Dodatni poeni

Za aktivnost i angažovanje na nastavi moguće je ostvariti **do 5 dodatnih poena**. Dodatni poeni služe isključivo kao dopuna projektnog rezultata do **maksimalnih 71 poena** i **ne mogu povećati projektni rezultat preko tog praga**.

3. Kontrolne tačke

Tokom semestra realizuju se **dve kontrolne tačke**, svaka sa **najviše 10 poena**. Prva se održava **u terminu Vežbe 4**, a druga **u terminu Vežbe 8**. Ukupno se na kontrolnim tačkama može ostvariti **20 poena**, koji su već uključeni u maksimum projektnog dela od **71 poena**.

Kontrolna tačka	Termin	Oblast provere	Poeni
KT1	Termin Vežbe 4	Osnovna implementacija	10
KT2	Termin Vežbe 8	Završna demonstracija	10

KT1 — OSNOVNA IMPLEMENTACIJA (4. VEŽBA)

- Repozitorijum i pokretanje projekta; strukturiran Tapiz backlog i podela rada.
- Osnovni domenski model, poslovna pravila i API/contract granice za sve dodeljene celine.
- Najmanje jedan izvršiv funkcionalni tok i automatizovani test značajnog pravila.
- Dogovor sa zavisnim timovima ili kontrolisani stub kada zavisna komponenta nije spremna.

KT2 — ZAVRŠNA DEMONSTRACIJA (8. VEŽBA)

- Realizovani obavezni use-case-ovi svake dodeljene celine u odobrenom obimu.
- Pozitivni i negativni testovi, integracija, build/CI i pregledani Pull Request-ovi.
- Ponovljivo pokretanje sa demo podacima i demonstracija zaokruženog poslovnog procesa.
- Poštovanje vlasništva nad podacima, prava pristupa i tenant izolacije gde je primenljivo.

Kontrolne tačke u terminima Vežbe 4 i Vežbe 8 i termini odbrane projekta iz odeljka 6 predstavljaju odvojene organizacione obaveze. Tačna satnica kontrolnih tačaka prati raspored računarskih vežbi i objave predmeta.

4. Prisustvo i obaveze na računarskim vežbama

- **Prisustvo računarskim vežbama je obavezno!**
- Student koji tokom semestra ima **više od dva neopravdana izostanka** sa računarskih vežbi **gubi pravo na polaganje projekta**.
- Da bi izostanak bio evidentiran kao opravdan, odgovarajuću potvrdu potrebno je dostaviti u najkraćem mogućem roku na adresu danijel.jovanovic@uns.ac.rs.
- U poruci je potrebno navesti ime i prezime, broj indeksa i datum vežbe na koju se izostanak odnosi, kako bi opravdanje moglo pravilno da se evidentira.

5. Informacije o projektu

Projektni zadatak realizuje se u okviru zajedničkog softverskog proizvoda **PYXIS** - informacionog sistema za upravljanje data centrom. Detaljni funkcionalni i tehnički zahtevi definisani su aktuelnom projektnom specifikacijom, dok su u nastavku navedena pravila organizacije rada koja su obavezujuća za sve timove.

5.1. Tim i dodela projektne celine

- Tim čini **od 6 do 10 studenata**.
- Svakom timu predmetni asistent dodeljuje projektne celine za koje tim preuzima primarnu odgovornost u okviru zajedničkog sistema.
- Svi timovi rade nad **istim zajedničkim GitHub repozitorijumom**. Pristup repozitorijumu administrira predmetni asistent.
- Pristup se dodeljuje na osnovu GitHub korisničkog imena. Student je dužan da dostavi tačno korisničko ime kako bi bio dodat kao saradnik na repozitorijumu i kako bi njegov individualni doprinos bio vidljiv kroz istoriju izmena.

5.2. Način rada u repozitorijumu

- Razvojni tok obuhvata Tapiz zadatak, radnu (feature) granu, smislene commit zapise, Pull Request, CI proveru, pregled koda (code review) i završnu VERIFY/QA proveru pre označavanja zadatka kao završenog.
- **Direktan push na main granu nije dozvoljen.** Značajne promene moraju biti povezane sa odgovarajućom korisničkom pričom i njenom čeklistom, a Pull Request mora biti pregledan pre spajanja.
- Kada promena utiče na druge module ili funkcionalnost druge projektne celine, neophodna je koordinacija sa timom koji tu celinu održava.
- Referentna tehnologija za izradu je **.NET/C#**. Upotreba druge tehnologije moguća je uz obezbeđenu interoperabilnost sa ostatkom sistema.

5.3. Minimalna očekivanja od projektne celine

- Projektna celina **mora biti deo zajedničkog proizvoda** i imati jasno definisanu granicu i ugovor prema drugim celinama; izolovana mini-aplikacija koja se ne može integrisati sa sistemom ne smatra se prihvatljivom realizacijom.
- Očekuju se automatizovani testovi ključnih poslovnih pravila i slučajeva upotrebe, kao i integracioni, ugovorni (contract/cross-team) i regresioni testovi kada su relevantni za dodeljenu celinu.
- Kada domen to omogućava, tokom semestra očekuje se **najmanje jedna značajna integracija sa drugom projektnom celinom**.
- Dokumentacija mora obuhvatiti odgovornost celine, glavne slučajeve upotrebe, korisničke priče i čekliste, javne ugovore, značajne arhitektonske odluke (ADR), uputstvo za lokalno pokretanje i testiranje, reprezentativan demonstracioni scenario i poznata ograničenja.
- Lokalno pokretanje, izgradnja (build) i testiranje ne smeju zavisiti od nedokumentovanih ručnih koraka poznatih samo jednom članu tima.
- **Svaki student mora imati vidljiv individualni doprinos** i na odbrani biti sposoban da obrazloži sopstveni rad, najmanje jedan relevantan slučaj upotrebe, donetu dizajnersku odluku, pripadajuće testove i zavisnost prema drugoj celini kada ona postoji.

6. Termini odbrane projekta

Odbrana projekta organizuje se u **dva osnovna termina**. Student, odnosno tim, projekat brani u jednom od ponuđenih termina.

Termin	Period	Napomena
Predispitni termin (predrok)	Prva polovina decembra	Raniji termin za studente čiji je projekat spreman za odbranu.
Redovni termin	Poslednja nedelja zimskog semestra	Redovni termin odbrane projekta.

Tačan datum, satnica i organizacija odbrane biće blagovremeno objavljeni putem zvaničnih kanala komunikacije predmeta.

6.1. Odbrana projekta

- Na odbrani se proverava funkcionalnost projekta, ispunjenost kriterijuma bodovanja i razumevanje donetih arhitektonskih, tehničkih i organizacionih odluka.
- Svaki član tima mora biti u stanju da obrazloži svoj doprinos i relevantne delove zajedničkog rešenja. **Prisustvo članova tima čija se odbrana evidentira je obavezno.**
- Korišćenje AI alata je dozvoljeno u skladu sa zahtevima predmeta, ali **student ostaje odgovoran za proveru, razumevanje i obrazlaganje svakog dela rešenja koji predstavlja na odbrani**.

6.2. Teorijska pitanja na odbrani

U okviru odbrane projekta student odgovara i na teorijska pitanja koja obuhvataju teme obrađene na predmetu i njihovu primenu na realizovanom projektu. Pitanja su grupisana po tematskim oblastima u nastavku.

6.2.1. Zahtevi, backlog, Git i timski razvoj

1. Objasnite jedan User Story iz vašeg projekta: koja je uloga, cilj i vrednost koju donosi?
2. Kako se piše korisnička priča po šablonu „Kao [uloga], želim/hoću/mogu [radnju], kako bih [postigao cilj]“ i koja je uloga zasebne čekliste?
3. Pokažite jednu stavku backlog-a i objasnite kroz koje je faze prošla do statusa Done.
4. Zašto Git istorija treba da bude sledljiv zapis razvoja, a ne samo mehanizam za čuvanje koda?
5. Šta čini kvalitetnu commit poruku i po čemu se razlikuje od poruke poput "update" ili "fix"?
6. Objasnite jedan pull request iz vašeg repozitorijuma: šta rešava, šta menja, kako je verifikovan?
7. Da li ste imali Git konflikt tokom razvoja? Kako ste odlučili koje ponašanje treba da bude konačno?
8. Šta znači da je stavka backlog-a zaista završena (Definition of Done) u vašem projektu?

6.2.2. OOP, Clean Code, SOLID i Clean Architecture

1. Pokažite primer interfejsa u vašem projektu i objasnite koji ugovor on definiše.
2. Objasnite princip enkapsulacije na konkretnoj klasi - zašto stanje nije javno izmenljivo bez kontrole?
3. Za jednu klasu u projektu objasnite koja je njena jedina odgovornost (Single Responsibility Principle).
4. Dajte primer gde je Open/Closed princip primenjen - kako dodajete novo ponašanje bez izmene postojećeg koda?
5. Objasnite Liskov Substitution Principle na primeru iz vašeg koda - da li se svaka implementacija ponaša u skladu sa ugovorom?
6. Šta je Interface Segregation Principle i zašto širok interfejs može biti problem?
7. Objasnite Dependency Inversion Principle - da li vaš poslovni sloj zavisi od apstrakcija ili od konkretnih tehničkih detalja?
8. Nacrtajte/opišite slojeve Clean Architecture u vašem projektu i smer zavisnosti između njih.
9. Šta je composition root i gde se u vašem projektu nalazi registracija konkretnih zavisnosti?
10. Koji deo vašeg projekta bi najviše patio kada bi Domain sloj neposredno zavisio od baze podataka?

6.2.3. Poslovna logika i use-case sloj

1. Izaberite jedan use-case i objasnite njegov ulaz, tok i mogući ishod (uspeh i neuspeh).
2. Kako vaš entitet čuva sopstvene invarijante i sprečava nevažeće stanje?
3. Objasnite razliku između poslovnog (očekivanog) neuspeha i tehničke greške u vašem sistemu.
4. Pokažite primer gde ste umesto bool/null/generičkog izuzetka koristili eksplicitan rezultat operacije.
5. Koje zavisnosti vaš use-case servis dobija kroz Dependency Injection i zašto?
6. Objasnite na kom nivou se u vašem sistemu vrši validacija ulaza, a na kom poslovnih pravila.
7. Koja operacija u vašem projektu mora biti tretirana kao jedna atomska poslovna transakcija?
8. Koju arhitektonsku odluku ste promenili tokom razvoja i zašto?

6.2.4. Testabilni dizajn i automatizovano testiranje

1. Pokažite jedan test i objasnite Arrange–Act–Assert strukturu na njemu.
2. Objasnite razliku između pozitivnog, negativnog i graničnog test scenarija u vašem projektu.
3. Gde i zašto ste koristili Moq (ili sličan alat) za izolaciju zavisnosti?
4. Zašto testabilan dizajn (DI, uske granice) olakšava pisanje testova bez baze i mreže?
5. Koji deo vašeg sistema je testiran integracionim, a ne samo jediničnim testom, i zašto?
6. Šta pokrivenost koda (code coverage) govori, a šta ne govori o kvalitetu testova?
7. Da li ste neki bug prvo reprodukovali testom, a zatim ispravili? Opišite taj slučaj.
8. Šta je Git tag manual-core-baseline u vašem projektu i zašto je taj trenutak važan?

6.2.5. Integracija modula, ugovori i podaci

1. Koje module ima vaš projekat i koja je poslovna granica svakog od njih?
2. Koji modul poseduje koji podatak, i kako drugi moduli tom podatku pristupaju?
3. Objasnite ugovor između dva modula - šta klijent zna, a šta ne zna o drugoj strani?
4. Kada u vašem sistemu koristite sinhroni poziv, a kada poslovni događaj (event)? Zašto baš tako?
5. Dajte primer poslovnog događaja iz vašeg sistema i objasnite ko ga objavljuje, a ko reaguje na njega.
6. Kako vaš sistem rešava konkurentan pristup istom resursu od strane dva korisnika ili procesa?
7. Šta znači idempotentnost i gde je u vašem projektu potrebna (npr. ponovljen zahtev)?
8. Koji integracioni ili ugovorni test štiti granicu između dva vaša modula?

6.2.6. Kontrolisan razvoj uz AI podršku

1. Koje projektne instrukcije (npr. AI_INSTRUCTIONS.md) ste definisali i zašto baš te?
2. Pokažite jedan zapis iz AI_USAGE.md i objasnite koja je odluka doneta i kako je verifikovana.
3. Koju sugestiju AI alata ste prihvatili, a koju odbacili? Obrazložite oba slučaja.
4. Da li ste napravili sopstvenu proceduru (skill) za neki ponovljivi zadatak? Šta ona radi?
5. Ako ste koristili više agentskih uloga, koja je odgovornost i ograničenje svake od njih?
6. Zašto rezultat AI alata sam po sebi nije dovoljan dokaz da je promena ispravna?
7. Kada biste u razvoju vašeg projekta izabrali jednostavan tok (jedan agent) umesto više uloga?

6.2.7. MCP i povezivanje sa projektom

1. Da li vaš projekat ima MCP server? Koje resurse ili alate on izlaže?
2. Objasnite razliku između MCP resursa i MCP alata na konkretnom primeru iz vašeg projekta.
3. Koji stvarni razvojni signal (test, diff, status) vraća neki od vaših MCP alata?
4. Kako ste primenili princip najmanjih privilegija pri definisanju dozvoljenih operacija server-a?
5. Šta je prompt injection i kako biste (ili jeste) zaštitili agentski tok od nepouzdanog sadržaja?
6. Koji podatak ili tajnu vaš MCP server namerno ne izlaže i zašto?

6.2.8. Hooks, guardrails, evaluacije i završna provera kvaliteta

1. Koju determinističku proveru (hook) ima vaš projekat i na koji događaj se ona okida?
2. Objasnite razliku između instrukcije (preporuke modelu) i hook-a (izvršivog mehanizma).
3. Koji guardrail sprečava rizičnu operaciju u vašem projektu, i šta bi se desilo bez njega?
4. Pokažite jedan evaluacioni scenario (uključujući negativni) koji proverava agentsko ponašanje.
5. Ko je uradio peer QA pregled vašeg projekta i koji je konkretan nalaz ili potvrda zabeležena?
6. Koji deo vašeg razvojnog toka mora biti garantovan kodom, a ne samo napisan kao instrukcija?
7. Koji je najveći preostali rizik ili ograničenje vašeg projekta i zašto ste ga prihvatili takvog?

6.2.9. Opšta i integrativna pitanja

1. Provedite nas kroz ceo put jedne funkcionalnosti: od zahteva, preko koda i testa, do AI podrške ako je korišćena.
2. Koja odluka u projektu je bila najteža i zašto ste je doneli baš tako?
3. Šta biste uradili drugačije da sada ponovo krećete sa projektom?
4. Koji deo sistema ne razumete u potpunosti i zašto (iskreno, ne kao trik pitanje)?
5. Objasnite jednu analogiju između klasičnog principa softverskog inženjerstva i njegove primene u razvoju uz AI (npr. interfejs ↔ ugovor alata, SRP ↔ agentska uloga).

7. Popravni termin

U popravnom terminu student izrađuje **novi projekat samostalno**. **Projektni poeni iz prethodne realizacije ne prenose se na novi projekat.**

Broj poena ostvaren na novom projektu množi se **koeficijentom 0,8**. Na taj način maksimalan broj poena koji je moguće ostvariti na projektnom delu u popravnom terminu iznosi **56,8 poena**. Na tako dobijen rezultat primenjuju se opšta pravila o minimumu projektnog dela, upisu ocene i usmenom delu ispita.

8. Usmeni deo ispita i konačna ocena

Usmeni deo ispita obuhvata proveru razumevanja principa obrađenih na nastavi, arhitekture i implementacije projekta, testiranja, integracionih odluka i načina upotrebe AI alata.

- Rezultat usmenog dela može uvećati ukupan broj poena za **najviše 30**, odnosno umanjiti ga za **najviše 20 poena**.
- Za studenta koji sa **51 ili više projektnih poena** pristupi usmenom delu, rezultat usmenog dela **ulazi u konačan obračun poena**.
- Predmet je položen kada konačan broj poena iznosi **najmanje 51**.

Ocena	6	7	8	9	10
Bodovi	51-60	61-70	71-80	81-90	91-100

9. Odgovornost za rad

Od studenata se očekuje jasno razgraničenje sopstvenog rada, rada ostalih članova tima i sadržaja dobijenog upotrebom spoljnih alata. U slučaju neovlašćenog preuzimanja tuđeg rešenja, plagijata ili lažnog predstavljanja doprinosa, za projektni zadatak evidentira se **0 poena**.

Upotreba AI alata ne umanjuje obavezu studenta da razume i samostalno obrazloži rešenje. Na odbrani se kao relevantan dokaz rada mogu koristiti repozitorijum, istorija izmena, testovi, dokumentacija i evidencija upotrebe AI alata.

10. Komunikacija i organizacione informacije

- **Organizacione informacije, tačni termini kontrolnih tačaka i odbrana, kao i eventualne izmene rasporeda, objavljuju se putem zvaničnih kanala komunikacije predmeta.**
- Za dostavljanje potvrde o opravdanom izostanku koristi se adresa **danijel.jovanovic@uns.ac.rs**.